Process Synchronization and Child Process Monitoring Using waitpid().

Code –

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    pid_t pid;
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    // Create child process
    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    else if (pid == 0) {

        // Child process
        printf("\nChild Process Started\n");
        printf("Child PID: %d\n", getpid());

        // Simulate some work
        printf("Child is working...\n");
        sleep(3);

        printf("Child Process Completed\n");

        // Exit with status 0
        exit(0);
    }

    else {

        // Parent process
        printf("\nParent is waiting for child...\n");

        // Synchronize with child using waitpid()
        pid_t result = waitpid(pid, &status, 0);

        if (result == -1) {
            perror("waitpid failed");
            return 1;
        }

        // Monitor child termination
        if (WIFEXITED(status)) {

            printf("\nChild process terminated normally.\n");
            printf("Child PID: %d\n", result);
            printf("Child Exit Status: %d\n",
                   WEXITSTATUS(status));
        }

        else {
```

```
    else {

        // Parent process
        printf("\nParent is waiting for child...\n");

        // Synchronize with child using waitpid()
        pid_t result = waitpid(pid, &status, 0);

        if (result == -1) {
            perror("waitpid failed");
            return 1;
        }

        // Monitor child termination
        if (WIFEXITED(status)) {

            printf("\nChild process terminated normally.\n");
            printf("Child PID: %d\n", result);
            printf("Child Exit Status: %d\n",
                   WEXITSTATUS(status));
        }

        else {
            printf("\nChild process terminated abnormally.\n");
        }
    }

    printf("\nParent Process Completed\n");

    return 0;
}
```

Result:

```
ketana_sindhu@KetaZone: ~/
bheeshma@SINGAM:~/SKILL$ nano Skill7a.c
bheeshma@SINGAM:~/SKILL$ gcc Skill7a.c -o s7a
bheeshma@SINGAM:~/SKILL$ ./s7a
Parent Process Started
Parent PID: 1838

Parent is waiting for child...

Child Process Started
Child PID: 1839
Child is working...
Child Process Completed

Child process terminated normally.
Child PID: 1839
Child Exit Status: 0

Parent Process Completed
bheeshma@SINGAM:~/SKILL$
```

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

```
ketana_sindhu@KetaZone: ~/
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

#define MAX_PATH 4096
#define MAX_COMMAND 100

/* Check whether a file is executable */
bheeshma@SINGAM: *path) {

    struct stat fileInfo;

    if (stat(path, &fileInfo) != 0)
        return 0;

    /* Check if it is a regular file and executable */
    if (S_ISREG(fileInfo.st_mode) &&
        access(path, X_OK) == 0) {

        return 1;
    }

    return 0;
}

/* Search command in PATH */
bheeshma@SINGAM: *command) {

    char *path = getenv("PATH");

    if (path == NULL) {
        return NULL;
    }

    /* Make a copy because strtok modifies the string */
    char *pathCopy = strdup(path);

    if (pathCopy == NULL) {
        return NULL;
    }

    char *directory = strtok(pathCopy, ":");

    static char fullPath[MAX_PATH];

    while (directory != NULL) {

        snprintf(fullPath,
                 sizeof(fullPath),
                 "%s/%s",
                 directory,
                 command);

        if (isExecutable(fullPath)) {

            free(pathCopy);
            return fullPath;
        }
```

```
  GNU nano 8.7.1
                command);

        if (isExecutable(fullPath)) {

            free(pathCopy);
            return fullPath;
        }

        directory = strtok(NULL, ":");
    }

    free(pathCopy);

    return NULL;
}

int main() {

    char command[MAX_COMMAND];

    printf("Command Resolution Using PATH\n");
    printf("-------------------------------\n");

    /* Retrieve PATH variable */
    char *path = getenv("PATH");

    if (path == NULL) {

        printf("Error: PATH variable not found.\n");
        return 1;
    }

    printf("PATH variable retrieved successfully.\n\n");

    printf("Enter command to search: ");

    scanf("%99s", command);

    /* Search for executable */
    char *result = findCommand(command);

    if (result != NULL) {

        printf("\nCommand found!\n");
        printf("Command      : %s\n", command);
        printf("Executable   : %s\n", result);
        printf("Permissions  : Executable\n");
        printf("Status       : Command can be executed.\n");
    }

    else {

        printf("\nCommand not found!\n");
        printf("Command      : %s\n", command);
        printf("Status       : No executable found in PATH.\n");
    }

    return 0;
}
```

Result:

```
bheeshma@SINGAM:~/SKILL$ nano Skil          l7b.c
bheeshma@SINGAM:~/SKILL$ gcc Skill          7b.c -o s7b
bheeshma@SINGAM:~/SKILL$ ./s7b
Command Resolution Using PATH
-------------------------------
PATH variable retrieved successfully.

Enter command to search: ls

Command found!
Command      : ls
Executable   : /usr/bin/ls
Permissions  : Executable
Status       : Command can be executed.
bheeshma@SINGAM:~/SKILL$ ./s7b
Command Resolution Using PATH
-------------------------------
PATH variable retrieved successfully.

Enter command to search: gcc

Command found!
Command      : gcc
Executable   : /usr/bin/gcc
Permissions  : Executable
Status       : Command can be executed.
bheeshma@SINGAM:~/SKILL$ ./s7b
Command Resolution Using PATH
-------------------------------
PATH variable retrieved successfully.

Enter command to search: hello123

Command not found!
Command      : hello123
Status       : No executable found in PATH.
bheeshma@SINGAM:~/SKILL$
```